

گزارش کار تمرین کامپیوتری دوم درس سیگنال ها و سیستم ها

امیرحسین صباحی

810101556

بخش اول

P1.m

این فایل اصلی ترین کد پروژه است و ترتیب مراحل را اجرا می کند.

مرحله اول – انتخاب تصویر ورودی

در ابتدا با استفاده از دستور `uigetfile`، یک پنجره برای انتخاب تصویر باز می شود. کاربر باید تصویر پلاک مورد نظر خود را انتخاب کند. پس از انتخاب تصویر، با دستور `imread` تصویر خوانده شده و در محیط MATLAB نمایش داده می شود. این بخش انعطاف پذیری برنامه را بالا می برد تا با هر تصویر جدیدی بتوان آزمایش را انجام داد. در صورتی که کاربر فایلی انتخاب نکند یعنی `Cancel` را بزند، برنامه متوقف می شود تا خطایی رخ ندهد.

مرحله دوم – تغییر اندازه تصویر

برای یکسان سازی ابعاد ورودی و جلوگیری از مشکلاتی که ممکن است در تحلیل اندازه ها ایجاد شود، تصویر به اندازه ی ثابت 300 در 500 پیکسل تغییر داده می شود. با این کار

همه تصاویر در یک مقیاس ثابت پردازش خواهند شد و مراحل بعدی مانند باینری سازی و حذف نویز عملکرد بهتری خواهند داشت.

مرحله سوم – تبدیل تصویر رنگی به خاکستری

در این مرحله از تابع `mygrayfun` که توسط کاربر طراحی شده استفاده می شود. این تابع سه کانال رنگی `R`، `G` و `B` تصویر را گرفته و با ضرایب استاندارد ترکیب می کند تا شدت روشنایی هر پیکسل محاسبه شود. نتیجه این تابع تصویری تک کاناله است که مقادیر بین صفر و یک دارد. این مرحله باعث کاهش داده ها و ساده تر شدن پردازش های بعدی می شود.

مرحله چهارم – باینری سازی تصویر

در این قسمت از تابع `mybinaryfun` استفاده می شود. ورودی این تابع تصویر خاکستری و مقدار آستانه است (مثلاً 0.5). در خروجی تصویر به حالت سیاه و سفید در می آید، به طوری که پیکسل هایی که مقدارشان بالاتر از آستانه هستند برابر 1 (سفید) و پیکسل های با مقدار کمتر برابر 0 (سیاه) می شوند. این مرحله سبب می شود تا پس زمینه از کاراکترهای پلاک جدا گردد و شکل کلی حروف مشخص شود.

مرحله پنجم – حذف نویز و اجزای کوچک

در این مرحله از تابع `myremovecom` استفاده می شود. این تابع تمام اجزای متصل کوچک تر از مقدار `n` (مثلاً 80 پیکسل) را حذف می کند. این کار باعث حذف لکه ها، نقاط اضافی و نویزهای کوچک در تصویر باینری می شود و تنها بخش های بزرگ تر (که احتمالاً حروف یا اعداد هستند) باقی می ماند. بعد از این مرحله تصویر تمیزتر و قابل اعتمادتر خواهد بود.

مرحله ششم – تقسیم‌بندی نواحی کاراکترها

در این بخش از تابع `mysegmentation` برای شناسایی اجزای متصل استفاده می‌شود. این تابع مشابه تابع آماده `bwlabel` عمل می‌کند و به هر ناحیه‌ی متصل یک شماره اختصاص می‌دهد. خروجی آن شامل دو متغیر است `L`: (تصویر برچسب‌خورده) و `num` عداد کل اجسام شناسایی‌شده.

برای هر بخش، ویژگی‌هایی مانند موقعیت و اندازه‌ی کادر اطراف کاراکتر با استفاده از تابع `regionprops` به‌دست می‌آید. سپس مستطیل قرمزی در اطراف هر کاراکتر رسم می‌شود تا جداسازی بخش‌ها به صورت بصری مشخص شود.

مرحله هفتم – مقایسه نواحی با دیتابیس

در این قسمت هر کاراکتر استخراج‌شده از پلاک با نمونه‌های موجود در دیتابیس مقایسه می‌شود. برای این کار ابتدا تصویر کاراکتر با دستور `imcrop` از تصویر اصلی جدا می‌شود، سپس اندازه آن با دستور `imresize` به ابعاد مشخص (مثلاً 24×42) تغییر داده می‌شود. در مرحله مقایسه از تابع `corr2` استفاده می‌شود که میزان شباهت بین تصویر برش‌خورده و هر تصویر موجود در دیتابیس را محاسبه می‌کند. اگر مقدار همبستگی بیشتر باشد، به معنای شباهت بیشتر بین دو تصویر است. در نهایت کاراکتری که بیشترین همبستگی را دارد به‌عنوان خروجی آن بخش انتخاب می‌شود. با تکرار این روند برای تمام نواحی شناسایی‌شده، کل کاراکترهای پلاک استخراج می‌شوند.

مرحله هشتم – نمایش و ذخیره خروجی

در انتها تمامی حروف و اعداد شناسایی‌شده در یک رشته متنی کنار هم قرار می‌گیرند تا شماره‌ی کامل پلاک تشکیل شود. این رشته در محیط `MATLAB` نمایش داده شده و در یک فایل متنی با نام `plate_output.txt` ذخیره می‌شود تا نتیجه به صورت دائمی در دسترس باشد.

در ضمن، برای نمایش تصویری خروجی، مجدداً تصویر باینری همراه با مستطیل‌های قرمز دور هر کاراکتر رسم می‌شود تا تشخیص کاراکترها به صورت بصری قابل مشاهده باشد.

```
F:\matlab\p1\p1.m
1  clc; close all; clear;
2  load MAPSET;
3  totalletters = size(TRAIN,2);
4  [file, path] = uigetfile({'*.jpg;*.jpeg;*.png;*.bmp;*.tif'}, 'Choose a plate image');
5  if isequal(file,0) || isequal(path,0)
6      error('No image selected.');
```

```
7  end
8  img = imread(fullfile(path, file));
9  figure; subplot(1,2,1); imshow(img); title('Original');
```

```
10 img = imresize(img, [300 500]);
11 subplot(1,2,2); imshow(img); title('Resized 300x500');
```

```
12
13 gray = mygrayfun(img);
14 figure; imshow(gray,[]); title('Grayscale');
```

```
15
16 thr = 0.5;
17 BW = mybinaryfun(gray, thr);
18 BW = ~BW;
19 figure; imshow(BW); title('Binary');
```

```
20
21 figure;
22 subplot(1,3,1); imshow(BW); title('Input');
```

```
23 BW_clean = myremovecom(BW, 200);
24 subplot(1,3,2); imshow(BW_clean); title('After myremovecom(200)');
```

```
25 BG = myremovecom(BW_clean, 3000);
26 BW_fg = BW_clean & ~BG;
27 subplot(1,3,3); imshow(BW_fg); title('Foreground');
```

```
28
29 [L, num] = mysegmentation(BW_fg);
30
31 stats = regionprops(L, 'BoundingBox');
```

```
32 if isempty(stats)
33     error('No segments found.');
```

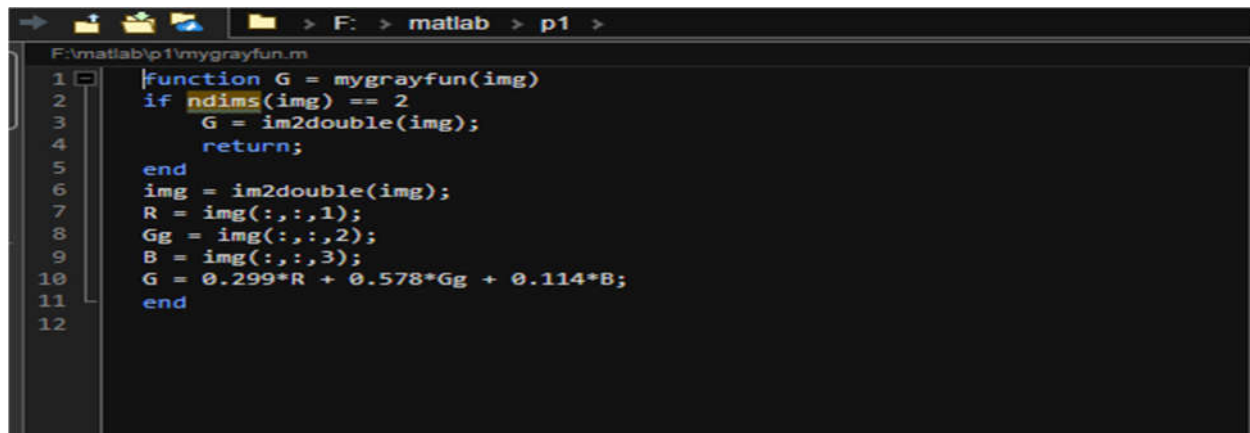
```
34 end
35 [~, sortIdx] = sort(arrayfun(@(s) s.BoundingBox(1), stats));
36
37 figure; imshow(BW_fg); title(sprintf('Segments = %d', num)); hold on;
```

```
38 for i = 1:numel(sortIdx)
39     rectangle('Position', stats(sortIdx(i)).BoundingBox, 'EdgeColor', 'r', 'LineWidth', 2);
```

```
F:\matlab\p1\p1.m
38 for i = 1:numel(sortIdx)
39     rectangle('Position',stats(sortIdx(i)).BoundingBox,'EdgeColor','g','LineWidth',2);
40 end
41 hold off;
42
43 final_output = [];
44 figure; colormap gray;
45 for idx = 1:numel(sortIdx)
46     n = sortIdx(idx);
47     [r, c] = find(L == n);
48     if isempty(r) || isempty(c), continue; end
49     Y = BW_fg(min(r):max(r), min(c):max(c));
50     Y = imresize(Y, [42 24]);
51     imagesc(Y); axis image off; title(sprintf('Segment %d',n)); drawnow;
52
53     ro = zeros(1, totalLetters);
54     for k = 1:totalLetters
55         ro(k) = corr2(TRAIN{1,k}, Y);
56     end
57     [MAXRO, pos] = max(ro);
58     if MAXRO > 0.45
59         ch = TRAIN{2, pos};
60         final_output = [final_output ch];
61     end
62 end
63
64 disp(['Plate: ', final_output]);
65 fid = fopen('number_Plate.txt','wt');
66 fprintf(fid, '%s\n', final_output);
67 fclose(fid);
68 winopen('number_Plate.txt');
69
70
```

Mygrayfun.m

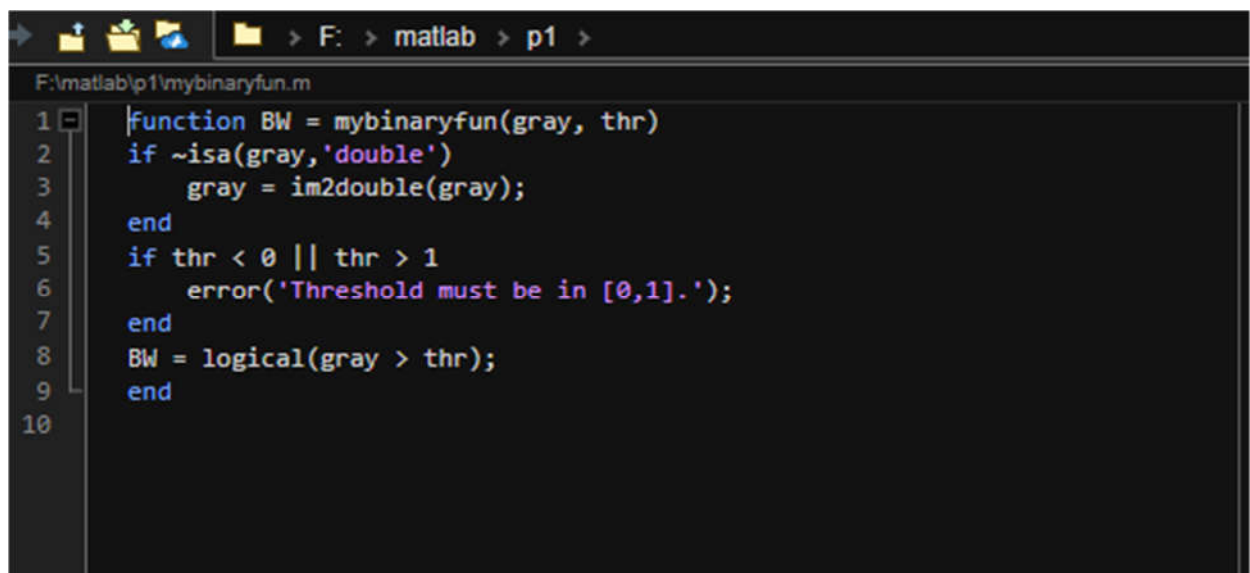
هدف این تابع تبدیل تصویر رنگی سه‌بعدی به تصویر خاکستری دوبعدی است. این تابع به‌جای استفاده از دستور آماده `rgb2gray` در MATLAB، به‌صورت دستی سه کانال رنگی قرمز، سبز و آبی را با ضرایب استاندارد ترکیب می‌کند تا شدت روشنایی پیکسل‌ها محاسبه شود.



```
F:\matlab\p1\mygrayfun.m
1 function G = mygrayfun(img)
2   if ndims(img) == 2
3       G = im2double(img);
4       return;
5   end
6   img = im2double(img);
7   R = img(:,:,1);
8   Gg = img(:,:,2);
9   B = img(:,:,3);
10  G = 0.299*R + 0.578*Gg + 0.114*B;
11  end
12
```

Mybinaryfun.m

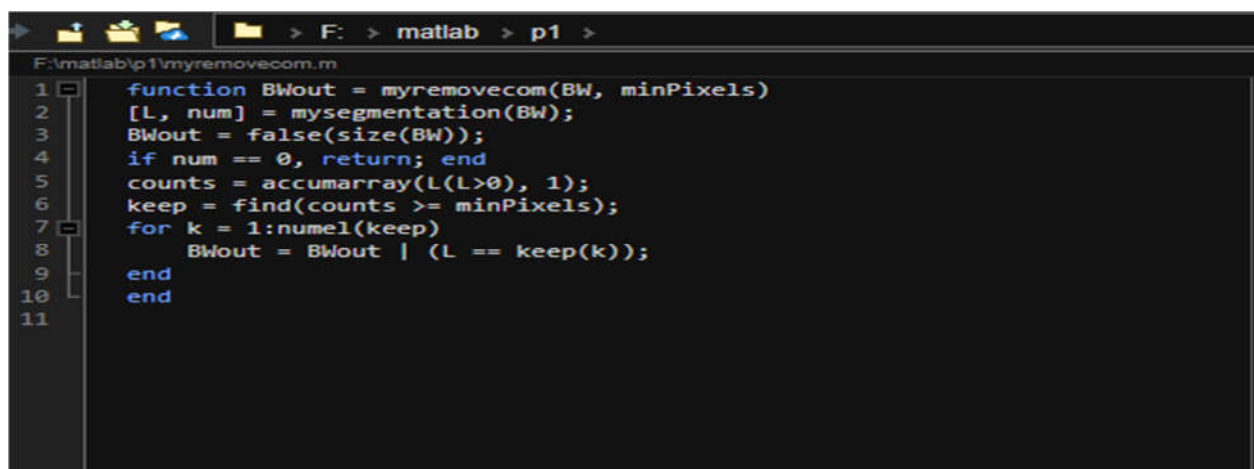
این تابع تصویر خاکستری را به تصویر باینری بر اساس آستانه‌ی ورودی تبدیل می‌کند. اگر شدت پیکسل از مقدار آستانه بیشتر باشد، مقدار آن ۱ در نظر گرفته می‌شود، در غیر این صورت ۰.



```
F:\matlab\p1\mybinaryfun.m
1 function BW = mybinaryfun(gray, thr)
2   if ~isa(gray, 'double')
3       gray = im2double(gray);
4   end
5   if thr < 0 || thr > 1
6       error('Threshold must be in [0,1].');
7   end
8   BW = logical(gray > thr);
9   end
10
```

Myremovecom.m

تابع `myremovecom` برای حذف اجزای کوچک و نویز از تصویر باینری استفاده می‌شود. ابتدا با کمک تابع `mysegmentation` نواحی متصل در تصویر شناسایی و شماره‌گذاری می‌شوند. سپس با استفاده از تابع `accumarray` تعداد پیکسل‌های هر ناحیه محاسبه می‌گردد. در ادامه فقط نواحی‌ای که تعداد پیکسل آن‌ها بیشتر از مقدار حداقل تعیین‌شده (`minPixels`) است انتخاب می‌شوند. در نهایت با ترکیب منطقی این نواحی، تصویر خروجی جدیدی ساخته می‌شود که تنها اجزای بزرگ و اصلی (مانند حروف و اعداد پلاک) در آن باقی مانده و تمام نویزها و بخش‌های کوچک حذف شده‌اند.



```
F:\matlab\p1\myremovecom.m
1 function BWout = myremovecom(BW, minPixels)
2 [L, num] = mysegmentation(BW);
3 BWout = false(size(BW));
4 if num == 0, return; end
5 counts = accumarray(L(L>0), 1);
6 keep = find(counts >= minPixels);
7 for k = 1:numel(keep)
8     BWout = BWout | (L == keep(k));
9 end
10 end
11
```

Mysegmentation.m

تابع `mysegmentation` برای شناسایی و شماره‌گذاری نواحی متصل (connected components) در تصویر باینری طراحی شده است. ورودی این تابع یک تصویر منطقی (دارای مقادیر 0 و 1) است. ابتدا یک ماتریس صفر به اندازه تصویر ایجاد می‌شود تا برچسب نواحی در آن ذخیره شود و شمارنده `num` برای نگه داشتن تعداد نواحی متصل از صفر شروع می‌کند. سپس الگوریتم به ترتیب تمام پیکسل‌ها را پیمایش می‌کند و هر زمان که به یک پیکسل سفید بدون برچسب برسد، شمارنده افزایش یافته و یک فرآیند جستجوی پشتگویی (stack-based flood fill) برای پیدا کردن تمام پیکسل‌های متصل به آن ناحیه اجرا می‌شود. جهت‌های مجاور در هشت راستا بررسی می‌شوند تا تمام پیکسل‌های مرتبط با آن ناحیه برچسب مشابه بگیرند. در پایان، خروجی `L` ماتریسی است که هر عدد در آن نشان‌دهنده‌ی شناسه یک ناحیه متصل است و خروجی `num` تعداد کل اجسام شناسایی‌شده را مشخص می‌کند. این تابع عملاً جایگزین تابع آماده `bwlabel` در MATLAB است و در پروژه برای تقسیم‌بندی حروف و اعداد پلاک استفاده می‌شود.

```
F:\matlab\p1 >
F:\matlab\p1\mysegmentation.m
1 function [L, num] = mysegmentation(BW)
2 assert(islogical(BW), 'Input must be logical.');
```

```
3 [m,n] = size(BW);
4 L = zeros(m,n,'uint32');
```

```
5 num = 0;
```

```
6 dirs = [-1 -1; -1 0; -1 1; 0 -1; 0 1; 1 -1; 1 0; 1 1];
```

```
7 for i = 1:m
8     for j = 1:n
9         if BW(i,j) && L(i,j)==0
10             num = num + 1;
```

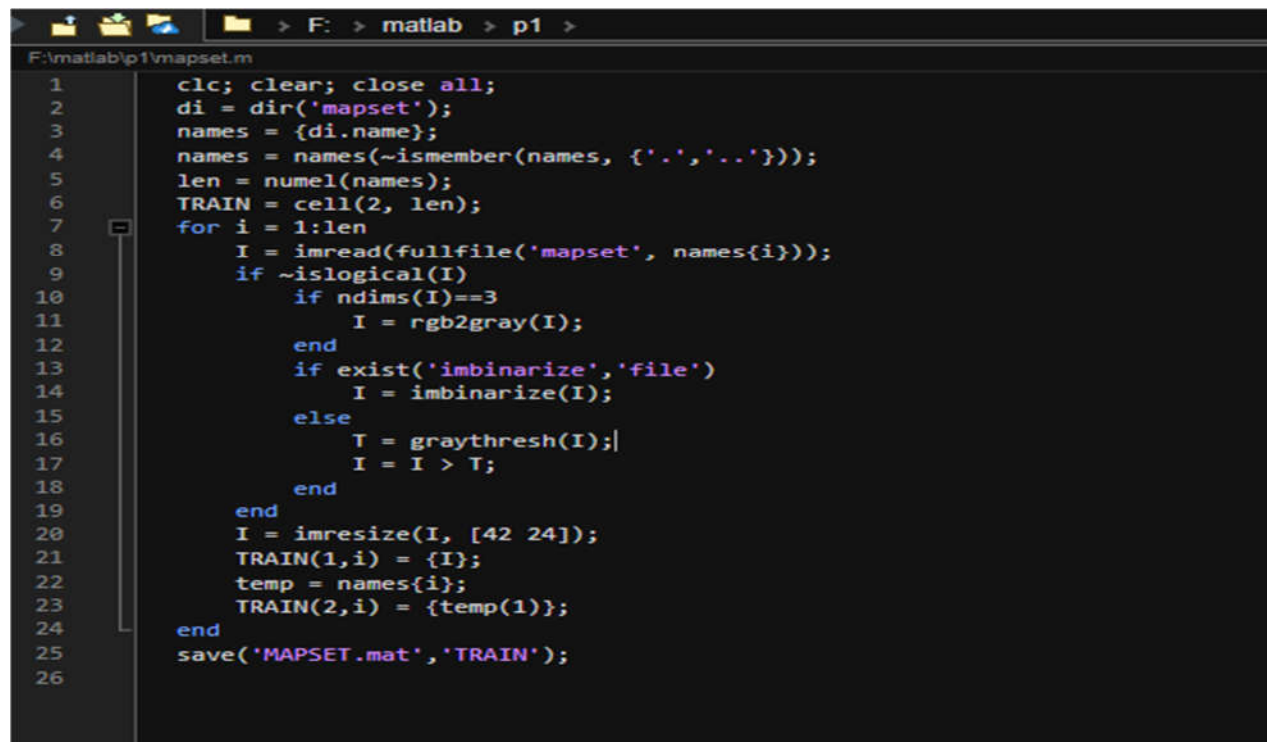
```
11             stack = [i j];
12             L(i,j) = num;
```

```
13             while ~isempty(stack)
14                 p = stack(end,:); stack(end,:) = [];
15                 for d = 1:8
16                     ii = p(1)+dirs(d,1);
17                     jj = p(2)+dirs(d,2);
18                     if ii>=1 && ii<=m && jj>=1 && jj<=n
19                         if BW(ii,jj) && L(ii,jj)==0
20                             L(ii,jj) = num;
```

```
21                             stack(end+1,:) = [ii jj];
22                         end
23                     end
24                 end
25             end
26         end
27     end
28 end
29 end
```


Mapset.m

این کد برای ساخت مجموعه داده آموزشی حروف و اعداد پلاک به صورت یک فایل mat استفاده می شود. ابتدا پوشه ای به نام mapset که شامل تصاویر حروف و اعداد است خوانده می شود و نام تمام فایل های آن استخراج می گردد. سپس برای هر تصویر عملیات آماده سازی انجام می شود؛ در صورتی که تصویر رنگی باشد با دستور rgb2gray به خاکستری تبدیل می شود و اگر باینری نباشد، با استفاده از تابع imbinarize یا در صورت نبود آن، با روش آستانه گذاری اتوماتیک graythresh به تصویر سیاه و سفید تبدیل می گردد. پس از آن اندازه تمامی تصاویر به 42 در 24 پیکسل تغییر داده می شود تا ابعاد ثابت برای کل مجموعه رعایت شود. هر تصویر آماده شده در سطر اول آرایه TRAIN و برچسب آن (اولین کاراکتر نام فایل) در سطر دوم ذخیره می شود. در پایان کل این داده ها در فایلی با نام MAPSET.mat ذخیره می شوند. این مجموعه در مراحل بعدی پروژه به عنوان دیتابیس اصلی برای تشخیص و مقایسه حروف و اعداد پلاک استفاده خواهد شد.



```
1 clc; clear; close all;
2 di = dir('mapset');
3 names = {di.name};
4 names = names(~ismember(names, {'.', '..'}));
5 len = numel(names);
6 TRAIN = cell(2, len);
7 for i = 1:len
8     I = imread(fullfile('mapset', names{i}));
9     if ~islogical(I)
10         if ndims(I)==3
11             I = rgb2gray(I);
12         end
13         if exist('imbinarize','file')
14             I = imbinarize(I);
15         else
16             T = graythresh(I);
17             I = I > T;
18         end
19     end
20     I = imresize(I, [42 24]);
21     TRAIN(1,i) = {I};
22     temp = names{i};
23     TRAIN(2,i) = {temp(1)};
24 end
25 save('MAPSET.mat', 'TRAIN');
26
```

Resized



Binary



Grayscale



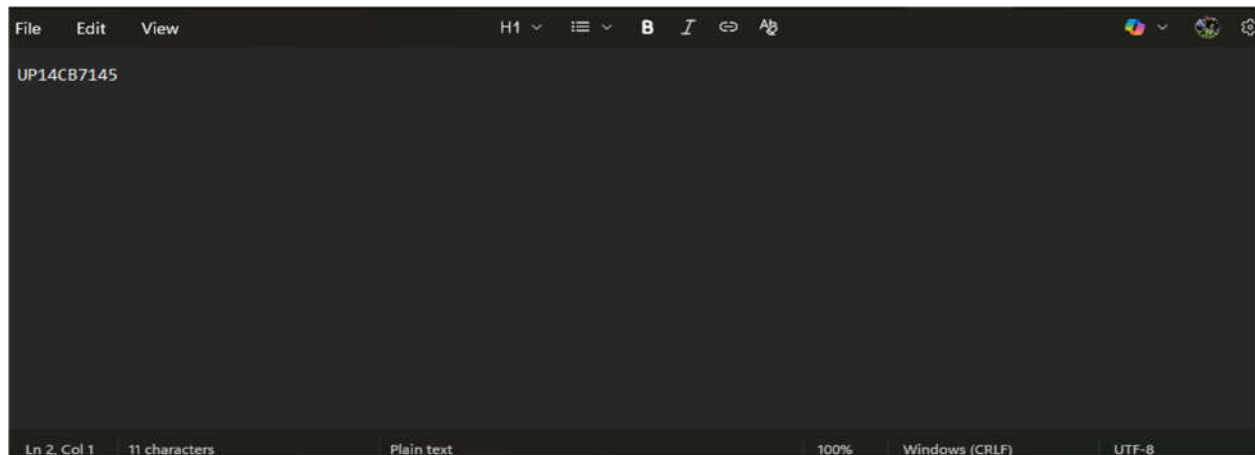
Myremovecom



Segmentation



Number_plate.txt



بخش دوم

P2.m

هدف این بخش پیاده‌سازی الگوریتمی برای استخراج حروف و اعداد پلاک خودروهای ایرانی با استفاده از پردازش تصویر است. در بخش اول پروژه از دیتابیس انگلیسی آماده استفاده شد، اما در این بخش لازم است دیتابیس فارسی شامل تمام حروف مورد نیاز پلاک‌های ایران (ج، س، ص، ط، ق، ل، م، ن، و، ه، ی) و همچنین تمام اعداد فارسی توسط کاربر ساخته شود. سپس با استفاده از روش‌های مشابه بخش اول، اعداد و حروف پلاک از تصویر استخراج و شناسایی می‌شوند.

توضیح کلی پروژه

ابتدا با دستور `uigetfile` تصویر پلاک خودرو انتخاب می‌شود. این کار باعث می‌شود کاربر بتواند هر تصویر دلخواهی را برای آزمایش برنامه انتخاب کند. پس از خواندن تصویر با دستور `imread`، برای یکنواخت‌سازی اندازه‌ها تصویر با `imresize` به ابعاد 300 در 500 پیکسل تغییر داده می‌شود. سپس تصویر به سطح خاکستری تبدیل می‌گردد تا شدت روشنایی پیکسل‌ها به دست آید. در این مرحله، به جای تابع اختصاصی بخش اول می‌توان از تابع آماده `rgb2gray` استفاده کرد زیرا در این بخش محدودیتی برای استفاده از توابع آماده وجود ندارد.

در مرحله بعد تصویر با دستور `imbinarize` یا با آستانه‌گذاری دستی به تصویر باینری (سیاه و سفید) تبدیل می‌شود. باینری‌سازی باعث می‌شود پس‌زمینه و کاراکترها از هم جدا شوند. سپس با استفاده از عملیات حذف نویز مانند `bwareaopen` نواحی کوچک و غیرضروری حذف می‌شوند تا فقط بخش‌های اصلی شامل حروف و اعداد باقی بمانند.

پس از تمیزسازی تصویر، مرحله تقسیم‌بندی (`segmentation`) انجام می‌شود تا هر ناحیه‌ی متصل به عنوان یک کاراکتر مجزا شناخته شود. با استفاده از تابع `regionprops` برای هر ناحیه، مختصات و اندازه‌ی مستطیل اطراف کاراکترها محاسبه می‌شود. سپس با دستور `rectangle` کادرهایی دور هر ناحیه کشیده می‌شود تا موقعیت کاراکترها مشخص گردد.

در مرحله بعد هر بخش جداشده از تصویر بریده (`imcrop`) و به اندازه استاندارد (مثلاً 24×42) تغییر داده می‌شود تا آماده مقایسه با داده‌های دیتابیس فارسی شود. برای مقایسه از معیار

همبستگی ($corr2$) استفاده می‌شود که شباهت بین تصویر کاراکتر و تصاویر دیتابیس را می‌سنجد. بالاترین مقدار همبستگی نشان‌دهنده کاراکتر صحیح است. در نهایت کاراکترهای تشخیص داده‌شده در یک رشته متنی کنار هم قرار گرفته و شماره پلاک استخراج می‌شود. خروجی نهایی شامل دو بخش است:

1. نمایش تصویری پلاک با کادر قرمز دور هر حرف و عدد.
2. چاپ رشته متنی شماره پلاک در محیط MATLAB و ذخیره آن در فایل `plate_output.txt`.

ساخت دیتابیس فارسی

از آنجا که در این بخش دیتابیس آماده وجود ندارد، لازم است کاربر دیتابیس را خود ایجاد کند. برای این منظور، از هر حرف و عدد فارسی یک تصویر باینری واضح در پوشه‌ای مانند "mapset" ذخیره می‌شود. سپس با استفاده از یک اسکریپت مشابه ساخت `MAPSET.mat` در بخش اول، تمامی تصاویر خوانده شده، اندازه آن‌ها یکنواخت می‌شود و در یک فایل `mat` به عنوان مجموعه داده آموزشی ذخیره می‌گردد. این مجموعه بعداً در شناسایی کاراکترها استفاده می‌شود.

نتیجه اجرای کد

اجرای اسکریپت `m2` باعث می‌شود تصویر پلاک ایرانی خوانده شود، ناحیه‌های مربوط به حروف و اعداد تفکیک شده و هر کاراکتر به کمک دیتابیس فارسی تشخیص داده شود. خروجی نهایی شماره کامل پلاک به صورت متنی چاپ می‌شود و در یک فایل متنی ذخیره می‌گردد. علاوه بر آن تصویر نهایی با کادرهای مشخص‌کننده نواحی حروف نمایش داده می‌شود.

جمع‌بندی

در این بخش الگوریتم شناسایی پلاک ایرانی با استفاده از همان اصول بخش اول اما با دیتابیس فارسی پیاده‌سازی شد. کاربر برای ساخت دیتابیس فارسی از تصاویر اختصاصی حروف و اعداد استفاده کرده و با آماده‌سازی مناسب داده‌ها توانسته است پلاک خودروهای ایرانی را از روی تصویر شناسایی کند. در این بخش مجاز به استفاده از توابع آماده MATLAB بوده‌ایم و نتیجه نهایی شامل تصویر پردازش‌شده و رشته متنی پلاک خودرو است. این مرحله پایه‌ی بخش سوم پروژه است که در آن ناحیه‌ی پلاک از تصویر کامل جلوی خودرو جدا می‌شود و همان مراحل تشخیص روی آن اعمال خواهد شد.

F:\matlab\p2\p2.m

```
1  clc; clear; close all;
2
3  [filename, pathname] = uigetfile({'*.jpg;*.png;*.jpeg'}, 'Select Plate Image');
4  if isequal(filename,0)
5      return;
6  end
7
8  img = imread(fullfile(pathname, filename));
9  img = imresize(img, [300 500]);
10 gray = rgb2gray(img);
11 bw = imbinarize(gray);
12 bw = imcomplement(bw);
13 bw = bwareaopen(bw, 80);
14 |
15 [L, num] = bwlabel(bw);
16 stats = regionprops(L, 'BoundingBox');
17
18 if num == 0
19     error('No characters detected.');
```

20

end

21

22

[~, idx] = sort(arrayfun(@(x)x.BoundingBox(1), stats));

23

stats = stats(idx);

24

25

db_files = dir(fullfile('database', '*.png'));

26

if isempty(db_files)

27

error('No database images found.');

28

end

29

30

db = cell(1, numel(db_files));

31

db_names = strings(1, numel(db_files));

32

33

for k = 1:numel(db_files)

34

fpath = fullfile('database', db_files(k).name);

```

34     fpath = fullfile('database', db_files(k).name);
35     t = imread(fpath);
36     if size(t,3)==3, t = rgb2gray(t); end
37     if ~islogical(t), t = imbinarize(t); end
38     db{k} = im2double(imresize(t, [42 24]));
39     [~, name, ~] = fileparts(fpath);
40     db_names(k) = string(name);
41 end
42
43 result = strings(1, num);
44
45 for i = 1:num
46     bbox = stats(i).BoundingBox;
47     ch = imcrop(bw, bbox);
48
49     if isempty(ch)
50         continue;
51     end
52
53     ch = imresize(ch, [42 24]);
54     ch = im2double(ch);
55
56     max_corr = -inf;
57     best = "";
58
59     for k = 1:numel(db)
60         db_img = db{k};
61         [r1, c1] = size(ch);
62         [r2, c2] = size(db_img);
63         if r1 ~= r2 || c1 ~= c2
64             db_img = imresize(db_img, [r1 c1]);
65         end
66         try
67             val = corr2(ch, db_img);

```



```

66         try
67             val = corr2(ch, db_img);
68         catch
69             continue;
70         end
71         if val > max_corr
72             max_corr = val;
73             best = db_names(k);
74         end
75     end
76
77     result(i) = best;
78 end
79
80 plate_str = join(result, '');
81 disp(['Recognized Plate: ', plate_str]);
82 fid = fopen('plate_output.txt', 'w');
83 fprintf(fid, '%s', plate_str);
84 fclose(fid);
85
86 figure('Name','Plate Recognition Result','NumberTitle','off');
87 imshow(img);
88 title(['Recognized Plate: ', plate_str], 'FontSize',14, 'Color','b');
89 hold on
90 for i = 1:num
91     rectangle('Position', stats(i).BoundingBox, 'EdgeColor', 'r', 'LineWidth', 1);
92 end
93 hold off
94

```

Result



بخش سوم

P3.m

هدف

در این بخش هدف تشخیص خودکار ناحیه پلاک خودرو از تصویر جلوی خودرو و سپس استخراج و شناسایی کاراکترهای پلاک است. در دو بخش قبلی کاراکترهای پلاک به صورت مستقیم از تصویر آماده پلاک استخراج می‌شدند، اما در این بخش برنامه باید ابتدا محل دقیق پلاک را از تصویر اصلی خودرو پیدا کند و پس از برش آن، همان مراحل پردازش و شناسایی کاراکترها را اجرا کند.

توضیح کلی کد

در ابتدای برنامه با دستور `uigetfile` از کاربر خواسته می‌شود تا تصویری از جلوی خودرو انتخاب کند. سپس تصویر خوانده شده و در محیط `MATLAB` نمایش داده می‌شود. در مرحله بعد تصویر رنگی به تصویر خاکستری تبدیل می‌گردد تا بتوان ویژگی‌های لبه‌ها را استخراج کرد. با استفاده از الگوریتم `Sobel` لبه‌های تصویر محاسبه می‌شوند. از آنجا که ناحیه پلاک معمولاً دارای خطوط افقی و عمودی مشخصی است، استفاده از لبه‌گیری باعث برجسته شدن آن در بین سایر نواحی خودرو می‌شود.

برای اتصال بخش‌های جداشده لبه‌ها از عملگر مورفولوژیکی `imclose` با عنصر ساختاری مستطیلی استفاده شده است تا نواحی شکسته به هم متصل شوند. سپس با دستور `imfill` نواحی بسته پر می‌شوند و با `bwareaopen` اجزای کوچک حذف می‌گردند تا فقط نواحی بزرگ‌تر مانند قاب پلاک باقی بمانند. با استفاده از تابع `regionprops` ویژگی‌هایی مانند مساحت و مختصات ناحیه‌ها استخراج می‌شود. برای تشخیص ناحیه پلاک از بین این اجسام، نسبت طول به ارتفاع و مساحت هر ناحیه بررسی می‌شود. پلاک

معمولاً مستطیلی با نسبت طول به ارتفاع بین ۲ تا ۵ و مساحتی نسبتاً بزرگ است. با استفاده از این معیارها ناحیه‌های احتمالی پلاک انتخاب می‌شوند.

در صورتی که چند ناحیه‌ی مشابه وجود داشته باشد، بزرگ‌ترین ناحیه بر اساس مساحت به‌عنوان ناحیه‌ی اصلی پلاک در نظر گرفته می‌شود. سپس با دستور `imcrop` ناحیه انتخاب‌شده از تصویر اصلی بریده و در اندازه 300 در 500 پیکسل بازسازی می‌شود. در ادامه همان مراحل پردازش بخش‌های قبل تکرار می‌شود. تصویر برش‌خورده به خاکستری تبدیل و باینری‌سازی می‌شود تا پس‌زمینه از حروف جدا گردد. پس از آن با استفاده از `bwareaopen` اجزای کوچک حذف شده و با تابع `bwlabel` و `regionprops` مختصات حروف و اعداد به دست می‌آید. کاراکترها بر اساس موقعیت افقی‌شان مرتب می‌شوند تا به ترتیب صحیح خوانده شوند.

برای تشخیص هر کاراکتر، ابتدا ناحیه مربوطه با `imcrop` استخراج و به اندازه استاندارد (24×42 پیکسل) تغییر داده می‌شود. سپس تصویر حاصل با تمام تصاویر موجود در دیتابیس مقایسه می‌شود. معیار مقایسه همبستگی (`corr2`) است که شباهت بین دو تصویر را اندازه‌گیری می‌کند. برای هر ناحیه، تصویری که بیشترین مقدار همبستگی را دارد به‌عنوان کاراکتر شناسایی شده انتخاب می‌شود. در پایان تمام کاراکترها کنار هم قرار گرفته و شماره کامل پلاک تشکیل می‌شود.

نتیجه نهایی در MATLAB شامل دو بخش است: چاپ رشته متنی پلاک شناسایی‌شده و نمایش تصویر اصلی خودرو با مستطیل قرمز دور ناحیه‌ی پلاک و عنوانی که شماره پلاک روی آن نوشته شده است. علاوه بر آن، نتیجه‌ی متنی در فایل `plate_output.txt` ذخیره می‌شود تا در صورت نیاز بتوان آن را به‌صورت جداگانه مشاهده کرد.

جمع‌بندی

در این بخش برنامه توانست ناحیه پلاک خودرو را به‌صورت خودکار از تصویر اصلی شناسایی و جدا کند. پس از برش ناحیه پلاک، همان الگوریتم شناسایی حروف و اعداد روی آن اعمال شد. استفاده از ویژگی‌های هندسی مانند نسبت طول به ارتفاع و مساحت باعث شد پلاک در میان سایر اجزای خودرو به‌درستی تشخیص داده شود. خروجی نهایی شامل تصویر خودرو با پلاک شناسایی‌شده و رشته متنی شماره پلاک است. این بخش گام مهمی در تکمیل سیستم جامع شناسایی پلاک خودرو محسوب می‌شود، زیرا قابلیت شناسایی از تصویر واقعی را فراهم می‌کند.

```
F:\matlab\p3\p3.m
1  clc; clear; close all;
2
3  [filename, pathname] = uigetfile({'*.jpg;*.png;*.jpeg'}, 'Select Car Front Image');
4  if isequal(filename,0)
5      return;
6  end
7
8  img = imread(fullfile(pathname, filename));
9  figure, imshow(img);
10
11  gray = rgb2gray(img);
12  edges = edge(gray, 'Sobel');
13  se = strel('rectangle', [15 15]);
14  closed = imclose(edges, se);
15  closed = imfill(closed, 'holes');
16  closed = bwareaopen(closed, 200);
17
18  stats = regionprops(closed, 'BoundingBox', 'Area');
19  possiblePlates = [];
20
21  for i = 1:length(stats)
22      bb = stats(i).BoundingBox;
23      aspectRatio = bb(3) / bb(4);
24      if aspectRatio > 2 && aspectRatio < 5 && stats(i).Area > 5000
25          possiblePlates = [possiblePlates; bb];
26      end
27  end
28
29  if isempty(possiblePlates)
30      error('No plate-like region detected.');
```

```

44 plateImg = imresize(plateImg, [300 500]);
45 gray = rgb2gray(plateImg);
46 bw = imbinarize(gray);
47 bw = imcomplement(bw);
48 bw = bwareaopen(bw, 80);
49
50 [L, num] = bwlabel(bw);
51 stats = regionprops(L, 'BoundingBox');
52 [~, idx] = sort(arrayfun(@(x)x.BoundingBox(1), stats));
53 stats = stats(idx);
54
55 db_files = dir(fullfile('database', '*.png'));
56 if isempty(db_files)
57     error('No database images found.');
```

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

|

-

```

58 end
59
60 db = cell(1, numel(db_files));
61 db_names = strings(1, numel(db_files));
62 for k = 1:numel(db_files)
63     fpath = fullfile('database', db_files(k).name);
64     t = imread(fpath);
65     if size(t,3)==3, t = rgb2gray(t); end
66     if ~islogical(t), t = imbinarize(t); end
67     db{k} = im2double(imresize(t, [42 24]));
68     [~, name, ~] = fileparts(fpath);
69     db_names(k) = string(name);
70 end
71
72 result = strings(1, num);
73 for i = 1:num
74     bbox = stats(i).BoundingBox;
75     ch = imcrop(bw, bbox);
76     if isempty(ch)
77         continue;
78     end
79     ch = imresize(ch, [42 24]);
80     ch = im2double(ch);
81     max_corr = -inf;
82     best = "";
83     for k = 1:numel(db)
84         db_img = db{k};
85         [c1, c1] = size(ch);

```

```

85         [r1, c1] = size(ch);
86         [r2, c2] = size(db_img);
87         if r1 ~= r2 || c1 ~= c2
88             db_img = imresize(db_img, [r1 c1]);
89         end
90         try
91             val = corr2(ch, db_img);
92         catch
93             continue;
94         end
95         if val > max_corr
96             max_corr = val;
97             best = db_names(k);
98         end
99     end
100     result(i) = best;
101 end
102
103 plate_str = join(result, '');
104 disp(['Recognized Plate: ', plate_str]);
105 fid = fopen('plate_output.txt', 'w');
106 fprintf(fid, '%s', plate_str);
107 fclose(fid);
108
109 figure('Name','Final Result','NumberTitle','off');
110 imshow(img);
111 title(['Detected Plate: ', plate_str],'FontSize',14,'Color','b');
112 hold on
113 rectangle('Position', plateBB, 'EdgeColor','r','LineWidth',2);
114 hold off
115

```

Result



بخش چهارم

P4.m

هدف

هدف این بخش طراحی الگوریتمی برای محاسبه سرعت متوسط یک خودرو با استفاده از پردازش ویدیو در محیط MATLAB است. در این مرحله سیستم باید بتواند از روی تصاویر متوالی ویدیو، جابه‌جایی خودرو را در طول زمان تشخیص دهد و با محاسبه فاصله طی شده و زمان سپری شده، سرعت متوسط خودرو را به دست آورد.

توضیح کلی کد

در ابتدا با دستور `uigetfile` از کاربر خواسته می‌شود تا فایل ویدیویی مورد نظر انتخاب شود. سپس با استفاده از تابع `VideoReader` ویدیو خوانده می‌شود و اطلاعاتی مانند نرخ فریم (`fps`) و مدت زمان ویدیو استخراج می‌گردد. فریم اول ویدیو به عنوان مرجع در نظر گرفته می‌شود و تصویر آن به خاکستری تبدیل می‌شود تا بتوان تغییرات بین فریم‌ها را به صورت دقیق محاسبه کرد.

در حلقه‌ی اصلی برنامه تمام فریم‌های ویدیو به ترتیب خوانده می‌شوند. برای هر فریم جدید، تصویر خاکستری آن با تصویر خاکستری فریم قبلی مقایسه می‌شود و اختلاف مطلق پیکسل‌ها با دستور `imabsdiff` محاسبه می‌گردد. این اختلاف بیانگر نواحی متحرک در تصویر است؛ در واقع مناطقی که مقدار آن‌ها تغییر کرده مربوط به حرکت خودرو هستند. سپس با دستور `imbinarize` تصویر اختلافی به باینری تبدیل می‌شود تا نواحی متحرک مشخص شوند و با `bwareaopen` اجزای کوچک و نویزها حذف می‌شوند تا فقط ناحیه بزرگ‌تر مربوط به خودرو باقی بماند.

با استفاده از تابع `regionprops` ویژگی‌های ناحیه‌های متصل محاسبه می‌شود. در هر فریم، ناحیه‌ای که بیشترین مساحت را دارد (بزرگ‌ترین ناحیه متحرک) به عنوان موقعیت خودرو در نظر گرفته می‌شود. مختصات مرکز افقی ناحیه (`center_x`) محاسبه و در آرایه `positions` ذخیره می‌شود. در فریم‌هایی که خودرو تشخیص داده نمی‌شود، مقدار `NaN` ثبت شده و در پایان با تابع `fillmissing` مقادیر خالی با روش درونیابی جایگزین می‌شوند تا مسیر حرکت خودرو پیوسته گردد.

پس از پایان پردازش ویدیو، زمان معادل هر فریم بر اساس نرخ فریم محاسبه می‌شود. با کم کردن موقعیت اولیه از سایر موقعیت‌ها، میزان جابه‌جایی خودرو در هر لحظه بر حسب پیکسل به دست می‌آید. با فرض این که هر پیکسل معادل 0.1 متر است تعیین شده توسط پارامتر (`scale_factor`)، فاصله طی شده بر حسب متر محاسبه می‌شود. سپس با تقسیم فاصله بر زمان، سرعت متوسط بر حسب متر بر ثانیه به دست می‌آید و با ضرب در 3.6 به کیلومتر بر ساعت تبدیل می‌شود.

در انتها مقدار سرعت متوسط در محیط MATLAB نمایش داده می‌شود و نموداری از تغییر موقعیت خودرو نسبت به زمان رسم می‌گردد. در محور افقی زمان (بر حسب ثانیه) و در محور عمودی فاصله طی شده (بر حسب متر) نمایش داده می‌شود. این نمودار حرکت خودرو را در طول ویدیو نشان می‌دهد و در عنوان آن سرعت متوسط محاسبه شده درج شده است.

جمع‌بندی

در این بخش با استفاده از پردازش تصویر و تحلیل اختلاف فریم‌های متوالی، حرکت خودرو در ویدیو شناسایی و موقعیت آن در هر لحظه ثبت شد. با محاسبه فاصله طی شده در زمان مشخص، سرعت متوسط خودرو استخراج گردید. این بخش آخرین مرحله از پروژه است که در کنار مراحل قبلی (استخراج پلاک از تصویر و تشخیص کاراکترها) سیستم کاملی برای شناسایی پلاک و محاسبه سرعت خودرو ارائه می‌دهد.

F:\matlab\p4\p4.m

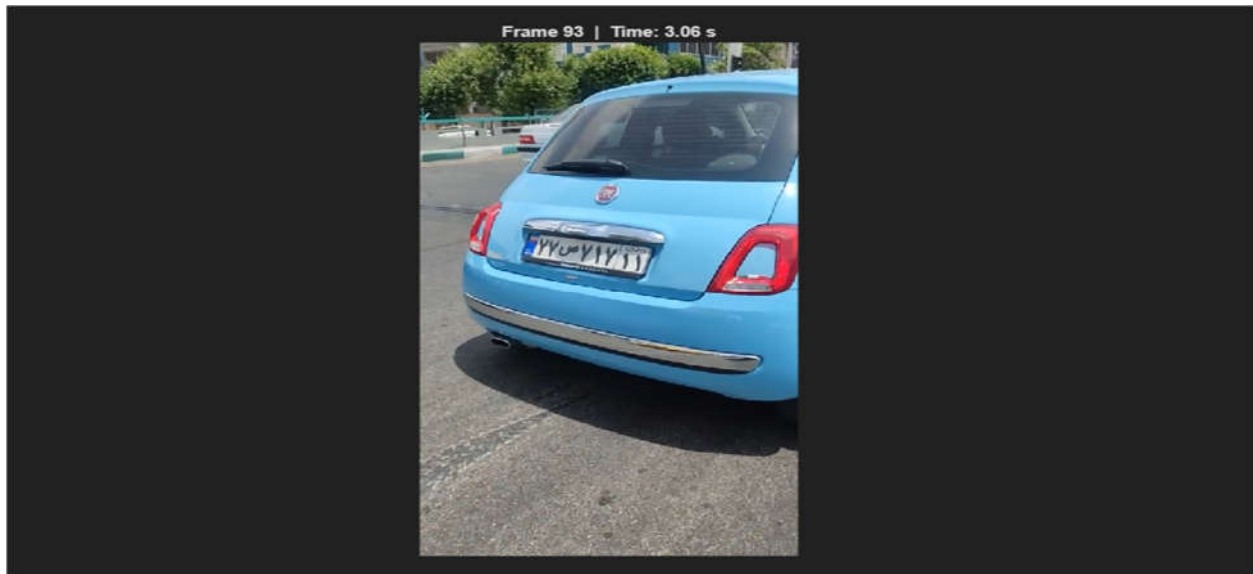
```
1  clc; clear; close all;
2
3  [filename, pathname] = uigetfile({'*.mp4;*.avi;*.mov'}, 'Select Video');
4  if isequal(filename,0)
5      return;
6  end
7
8  vid = VideoReader(fullfile(pathname, filename));
9  fps = vid.FrameRate;
10 frame_count = floor(vid.Duration * fps);
11
12 firstFrame = readFrame(vid);
13 gray_prev = rgb2gray(firstFrame);
14
15 positions = [];
16 frame_idx = 1;
17
18 while hasFrame(vid)
19     frame = readFrame(vid);
20     gray = rgb2gray(frame);
21     diff_img = imabsdiff(gray, gray_prev);
22     bw = imbinarize(diff_img);
23     bw = bwareaopen(bw, 200);
24     stats = regionprops(bw, 'BoundingBox', 'Area');
25     if ~isempty(stats)
26         [~, idx] = max([stats.Area]);
27         bbox = stats(idx).BoundingBox;
28         center_x = bbox(1) + bbox(3)/2;
```

```

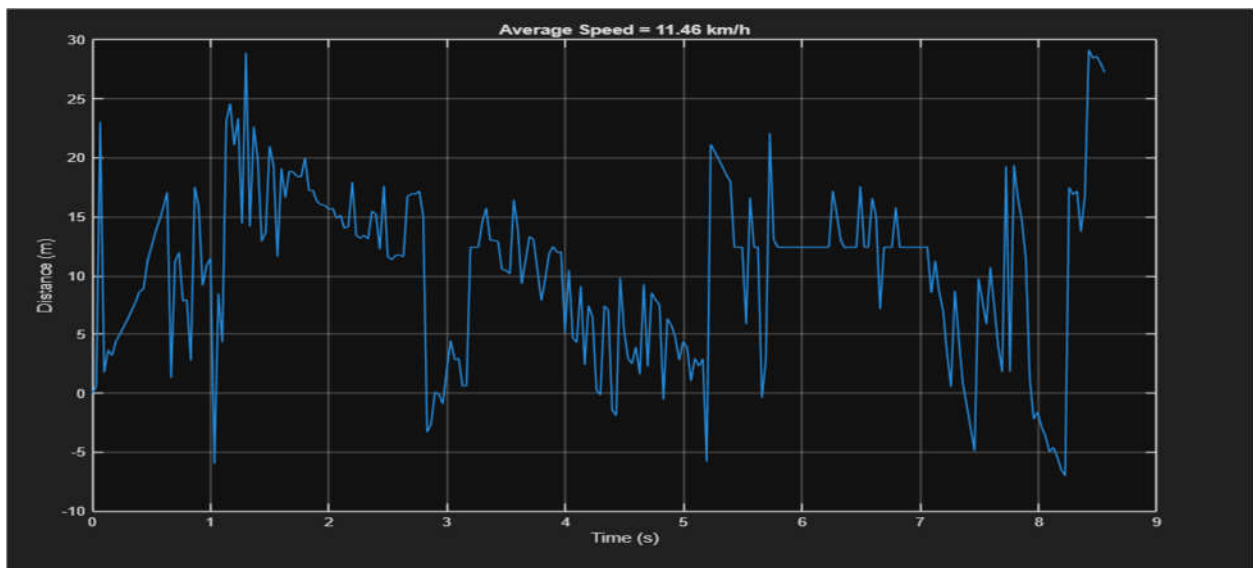
25     if ~isempty(stats)
26         [~, idx] = max([stats.Area]);
27         bbox = stats(idx).BoundingBox;
28         center_x = bbox(1) + bbox(3)/2;
29         positions(end+1) = center_x;
30     else
31         positions(end+1) = NaN;
32     end
33     gray_prev = gray;
34     frame_idx = frame_idx + 1;
35 end
36
37 positions = fillmissing(positions, 'linear');
38 time = (0:length(positions)-1) / fps;
39 dist_px = positions - positions(1);
40 scale_factor = 0.1;
41 dist_m = dist_px * scale_factor;
42
43 total_distance = dist_m(end);
44 total_time = time(end);
45 avg_speed_mps = total_distance / total_time;
46 avg_speed_kmh = avg_speed_mps * 3.6;
47
48 disp(['Average Speed: ', num2str(avg_speed_kmh, '%.2f'), ' km/h']);
49
50 figure('Name','Car Tracking','NumberTitle','off');
51 plot(time, dist_m);
52 xlabel('Time (s)');
53 ylabel('Distance (m)');
54 title(['Average Speed = ', num2str(avg_speed_kmh, '%.2f'), ' km/h']);
55 grid on;
56

```

Video



Average Speed



Command Window

New to MATLAB? See resources for [Getting Started](#)

Average Speed: 11.46 km/h

>>